

IS4102 Advanced Software Quality Assurance

QA Automation Practical Assignment

Test Automation Framework for automationexercise.com

Name	B.A.U.I. Balasooriya
Index Number	22020081
Application Under Test	https://automationexercise.com
GitHub Repository	https://github.com/UthumImasha/SQA_Assignment
Technology	Java 17, Selenium 4, TestNG, Maven, POM

1. Website Selection

For this assignment I selected **AutomationExercise** (<https://automationexercise.com>). Our lecturer allowed any demo website, and I first considered saucedemo.com because it is well known, but it felt too small once I started listing the scenarios I wanted to write. AutomationExercise behaves like a proper online store. It has registration and login, a product catalog with search, a cart, a checkout with a fake payment step, and a contact form, so there was enough material for more than 15 manual scenarios without repeating myself.

Another reason for the choice is that the site adds data-qa attributes to its important elements. I expected locators to be my biggest maintenance risk, and these attributes exist specifically for testers, so that settled it.

2. Requirement Analysis

2.1 Application Overview

AutomationExercise is a clothing store demo with products for women, men and kids. Since the whole site exists for QA practice, I could freely create accounts and place orders without affecting any real business.

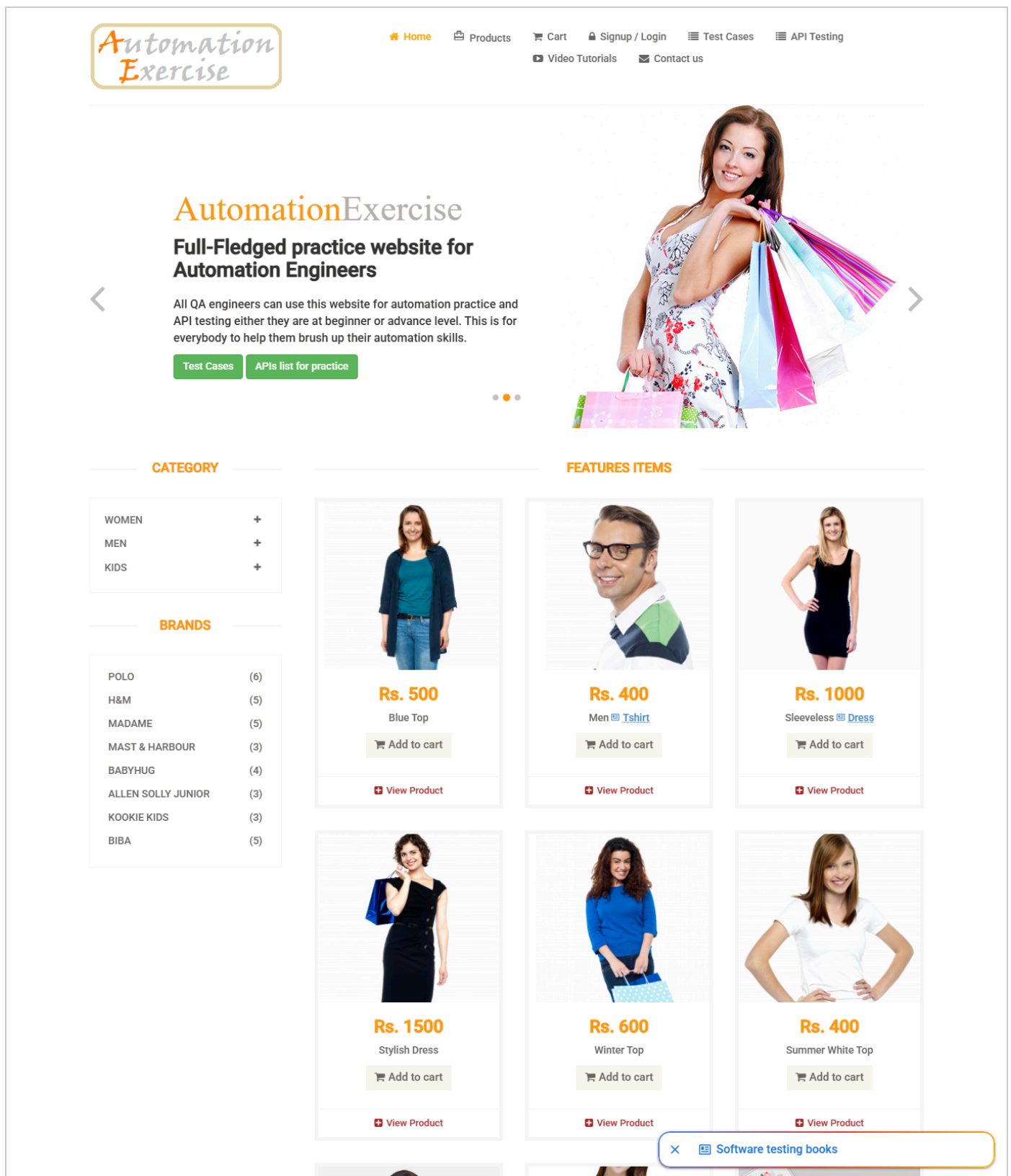


Figure 1: Home page with category sidebar, brand sidebar and featured products

2.2 Objectives

Before writing any scenarios I wanted to understand the main user flows, figure out which features are stable enough to automate, and collect reasons I could use later when justifying the automation candidates.

2.3 Target Users

User type	What they can do
Guest visitor	Browse and search products, view product details, add items to cart, subscribe to the newsletter, send a contact message
Registered customer	Everything a guest can do, plus login/logout, checkout and place orders, download an invoice, delete their own account

There is no admin side visible, so administration stays out of scope.

2.4 Key Features

Feature area	Details
Authentication	Signup with a full account information form, login with email and password, logout, account deletion. There is no email verification step.
Product catalog	34 products with names and prices, category sidebar (Women, Men, Kids), brand sidebar, product detail pages.
Search	Search box on the Products page. Results appear under a "Searched Products" heading.
Cart	Add from the listing or the detail page, quantity selection, line totals, item removal, checkout entry point.
Checkout and payment	Address review, order comment, a payment form that accepts any card details, order confirmation with invoice download.
Contact form	Name, email, subject, message, optional file upload, and a JavaScript confirmation dialog.
Newsletter	Subscription box in the footer of every page.

2.5 Assumptions

- The site stays available and unchanged during the assignment period
- Payment is simulated, so no real transaction can happen
- Creating and deleting test accounts is acceptable on a practice site
- Registered emails stay in a shared database, so every run needs a fresh unique email
- Testing is done in English on desktop Chrome

2.6 Risks and Constraints

Two of these risks actually hit me during development (the ad overlays and the slow responses), which I describe later in the debugging section.

Risk	Impact on testing	Mitigation
Third party ads overlay page elements	Automated clicks can land on an ad instead	Scroll the target into view and click the element directly
Shared public database that other testers also use	Existing data can change or vanish at any time	Each test creates its own account and data
Free demo site with no SLA	Occasional slowness causes timeouts	Explicit waits with sensible timeouts, no fixed sleeps
Duplicate email registration is rejected	Reused signup data breaks the test	Timestamped unique email for every run
Temporary downtime is possible	The whole suite fails for environmental reasons	Rerun later and document the dependency

2.7 Observations Relevant to Automation

- While clicking through the site I noticed that the important elements carry data-qa attributes, which became the base of my locator strategy
- Adding to cart opens a modal through AJAX, so waits are needed there
- Empty-field validation is done with HTML5 required attributes
- The layout has clearly not changed for years (the footer still says 2021), which lowers the risk of locators breaking mid-project

3. Manual Test Scenarios

I wrote 16 scenarios covering the main flows. Priorities follow business impact: anything on the register, login or purchase path is High, because a failure there directly blocks a customer from buying.

ID	Test Scenario	Expected Result	Priority
TS01	Register a new user with valid details and a unique email	Account created; user logged in automatically; header shows "Logged in as (username)"	High
TS02	Register with an already registered email	"Email Address already exist!" message	High
TS03	Login with valid credentials	User logged in; username in header	High
TS04	Login with invalid credentials	"Your email or password is incorrect!" message	High
TS05	Logout from a logged in session	Redirected to login page; session ended	Medium
TS06	Delete the logged in user's account	"Account Deleted!" confirmation	Medium
TS07	Search for an existing product	Matching products under "Searched Products"	High
TS08	Search for a product that does not exist	No products listed; no error	Medium
TS09	Filter products by category	Only that category's products shown	Medium
TS10	Filter products by brand	Only that brand's products shown	Low
TS11	Open a product's detail page	Name, category, price, availability, condition, brand shown	Medium
TS12	Add a product to the cart from the listing	"Added!" modal; cart shows correct name, price, quantity 1	High
TS13	Set quantity on detail page and add to cart	Cart line shows chosen quantity and correct total	Medium
TS14	Remove a product from the cart	Line disappears; cart updates	Medium
TS15	Checkout and place an order as a logged in user	Address matches registration; payment accepted; order confirmed	High
TS16	Submit the contact form with an attachment	Confirmation dialog, then success message	Medium

Automation
Exercise

[Home](#)[Products](#)[Cart](#)[Signup / Login](#)[Test Cases](#)[API Testing](#)[Video Tutorials](#)[Contact us](#)

Login to your account

Email Address

Password

Login

OR

New User Signup!

Name

Email Address

Signup

SUBSCRIPTION

Your email address

Get the most recent updates from our site and be updated your self...

×

User management software

Figure 2: Login and signup forms (TS01 to TS04)



Search Product



CATEGORY

[WOMEN](#) +
[MEN](#) +
[KIDS](#) +

BRANDS

[POLO](#) (6)
[H&M](#) (5)
[MADAME](#) (5)
[MAST & HARBOUR](#) (3)
[BABYHUG](#) (4)
[ALLEN SOLLY JUNIOR](#) (3)
[KOOKIE KIDS](#) (3)
[BIBA](#) (5)

ALL PRODUCTS



Rs. 500

Blue Top

[Add to cart](#)

[View Product](#)



Rs. 400

[Men](#) [Tshirt](#)

[Add to cart](#)

[View Product](#)



Rs. 1000

Sleeveless [Dress](#)

[Add to cart](#)

[View Product](#)

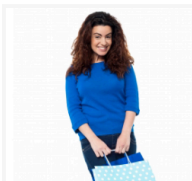


Rs. 1500

Stylish Dress

[Add to cart](#)

[View Product](#)



Rs. 600

Winter Top

[Add to cart](#)

[View Product](#)

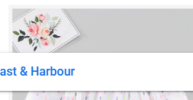
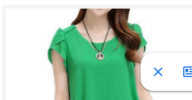


Rs. 400

Summer White Top

[Add to cart](#)

[View Product](#)



[Mast & Harbour](#)

Figure 3: Products page with search (TS07 to TS12)

4. Automation Decision

4.1 How I Chose

Five scenarios could be automated, so I scored the candidates against a few practical questions. Does it protect the money path? Will it be run again and again as a regression check? Does it have a clear, stable outcome my script can assert? And can Selenium actually do it without fighting captchas or making visual judgements? The five that scored best on those questions got the slots.

4.2 Selected for Automation

ID	Scenario	Why I picked it
TS01	User registration	The entry point for every customer. The signup form is long, so checking it manually every time is slow and error prone. My script generates a unique email per run and deletes the account at the end, so it cleans up after itself.
TS04	Invalid login	Protects the account security path. It needs no stored test data at all (a freshly generated email cannot belong to anyone), so other testers on the shared database can never break it.
TS07	Product search	The main way users find products. There is a clear results heading to assert, and later it can be extended with more keywords.
TS12	Add to cart	First step of the revenue path. Checking name, price and quantity against the listing is exactly the repetitive detail work where scripts beat humans.
TS15	Checkout order placement	The most valuable flow in the whole application. It chains login, cart, address and payment, so it catches integration breakage that page-level tests cannot see.

One decision I want to explain: I did not give valid login (TS03) its own slot. The checkout test has to log in as its very first step anyway, so the positive login path already runs on every suite execution. A separate valid-login script would test the same thing twice and waste a slot.

4.3 Not Selected, and Why

ID	Reason not selected
TS02	Needs a known existing account, but on this shared database any tester can delete it at any time, so the test would be unreliable
TS03	Already covered inside the checkout flow, as explained above
TS05	A single click with very low defect probability

TS06	Already runs as the cleanup step of the registration script
TS08	Low-risk negative case; better added later as an extra data row for the search script
TS09	Navigation only, and the product-to-category mapping belongs to the site, so assertions would be weak
TS10	Same reasoning as TS09, and it is the lowest priority scenario I have
TS11	Mostly about how the page looks, which a human judges better than string comparisons
TS13	Only a small variation of TS12; not worth a slot of its own
TS14	Useful, but the slots went to flows with direct revenue impact
TS16	Automatable (even the file upload), but it is a support flow with no revenue impact

5. Framework Design

5.1 The Idea

My main goal was that a test should read like its manual scenario. Tests hold only the flow and the assertions. Every page of the site gets one Java class that knows that page's locators and actions, and everything else (driver setup, waits, config, screenshots) sits in small supporting packages. When the UI changes, I fix one page class and every test that uses it is repaired at once.

```
tests (TC01..TC05, flow + assertions)
    | calls
pages (one class per page: private locators + public actions)
    | drives
Selenium WebDriver (Chrome via DriverFactory)
    |
automationexercise.com

supporting: base (driver lifecycle) | utils (config, waits, data, screenshots) | listeners
(failure hooks)
```

5.2 Project Structure and Packages

```
src/test/java/com/automationexercise/
├─ base/           DriverFactory.java, BaseTest.java
├─ pages/          HomePage, SignupLoginPage, SignupDetailsPage, AccountPage,
│                  ProductsPage, CartPage, CheckoutPage, PaymentPage
├─ tests/          TC01_RegistrationTest ... TC05_CheckoutTest
├─ utils/          ConfigReader, WaitUtils, ScreenshotUtil, TestDataGenerator
└─ listeners/      TestListener.java
src/test/resources/ config.properties, testng.xml
```

Package	What it does
base	DriverFactory builds a configured ChromeDriver. Selenium Manager fetches the matching driver binary by itself, so there is no chromedriver.exe in the project. BaseTest opens a fresh browser before every test method and quits it after, so no test can leak login state or cart items into the next one.
pages	One class per page. Locators are private By fields at the top, using data-qa attributes first, then ids, then stable CSS. The public methods are user actions like login() or addFirstProductToCart(). Tests never touch a locator.
tests	One TestNG class per selected scenario, with clear assertion messages so a failure explains itself.
utils	ConfigReader loads config.properties (base URL, browser, headless flag, timeout). WaitUtils wraps WebDriverWait; there is no Thread.sleep anywhere in the project. Its scrollAndClick() also scrolls the

	element to the centre of the screen first, which turned out to matter more than I expected (see the ad story in section 7). ScreenshotUtil saves timestamped PNGs and TestDataGenerator makes unique emails from the current time.
listeners	TestListener implements TestNG's ITestListener. When any test fails it saves a screenshot automatically. It is registered once in testng.xml, so test code never mentions screenshots.
resources	config.properties keeps environment settings out of the code. testng.xml defines the suite and Maven Surefire runs it through mvn test.

5.3 Design Decisions

Decision	Reason
Fresh browser per test method	Full test independence; any test can run alone or in any order
Explicit waits only, no sleeps	Fixed sleeps are always either too slow or too short on a demo site with uneven response times
data-qa first locator strategy	Those attributes are there for testers and survive styling changes
Config in a properties file	Changing URL or timeouts needs no code edit
Unique generated email per run	The shared database rejects duplicate registrations
Screenshot on failure via a listener	Every failure leaves visual evidence without cluttering the tests

6. Automation Implementation

All five scenarios are automated and passing. I kept a one-to-one mapping, one test class per scenario, so anyone can trace a script back to its scenario row.

Test class	Scenario	What it verifies
TC01_RegistrationTest	TS01	Registers with a generated unique email, asserts "Account Created!", checks the header shows the new username, then deletes the account and asserts "Account Deleted!"
TC02_InvalidLoginTest	TS04	Tries to log in with a freshly generated (so unregistered) email and asserts the exact error message
TC03_ProductSearchTest	TS07	Searches "dress", asserts the results heading, that results exist, and that they relate to the keyword
TC04_AddToCartTest	TS12	Remembers the name and price from the listing, adds the product through the AJAX modal, then asserts the cart shows the same name and price with quantity 1 and a matching total
TC05_CheckoutTest	TS15	Registers, fills the cart, checks the delivery address on the checkout page matches the registration data, pays with dummy card details, asserts the order confirmation, then deletes the account

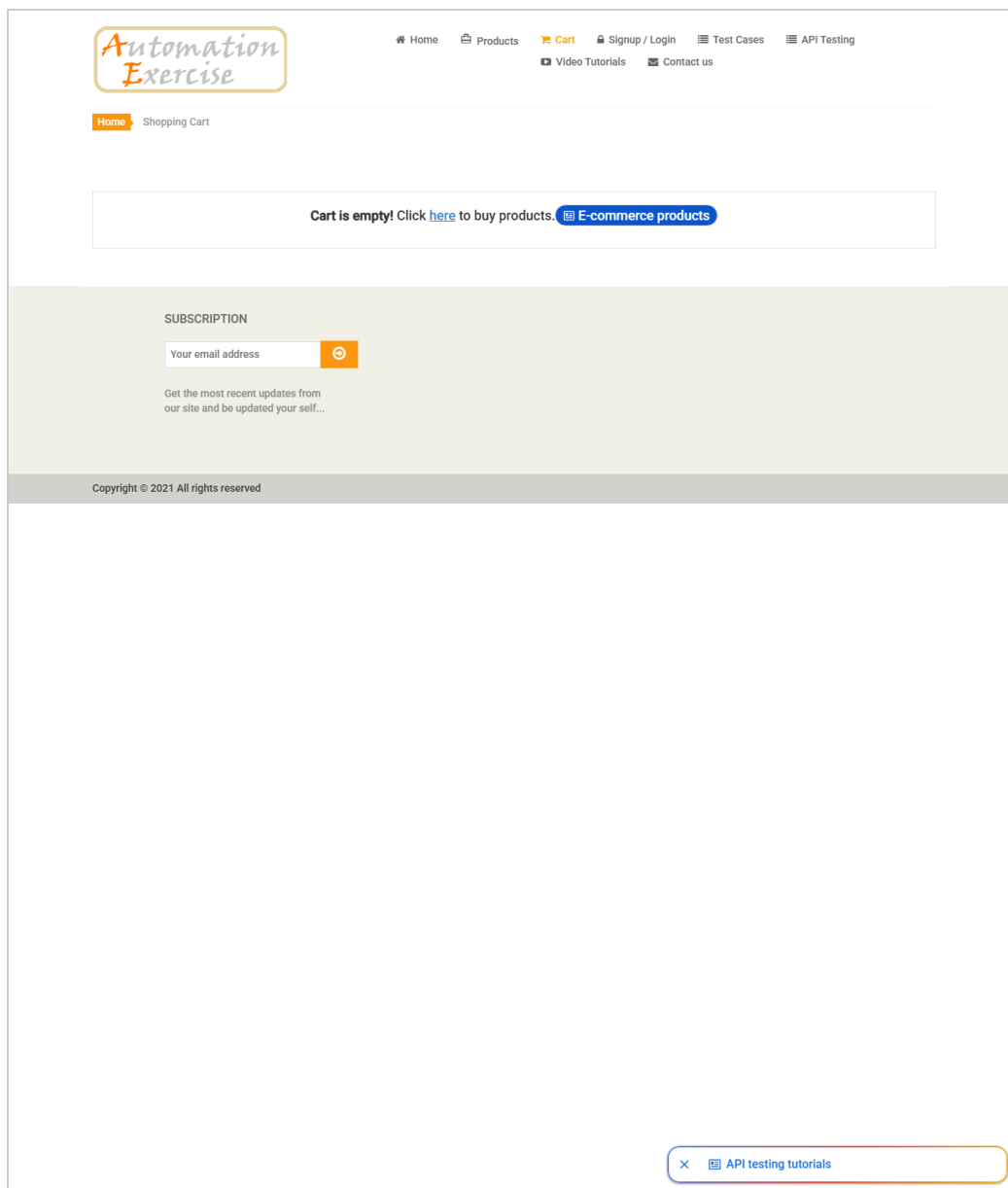


Figure 4: Cart page verified by TC04 (name, price, quantity, total)

6.1 Suite Result

```
[INFO] Running TestSuite
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 01:33 min
```

The suite runs with `mvn test` from the repository root and takes about a minute and a half on my machine. Chrome opens visibly by default; there is a headless switch in `config.properties`. No driver executable is needed anywhere because Selenium Manager downloads the right chromedriver for the installed Chrome on its own.

7. Debugging Challenge

7.1 The Intentional Failure

For this step I broke the framework on purpose. I removed the explicit wait inside `ProductsPage.goToCartFromModal()` and replaced it with a direct `findElement().click()` on the View Cart link. That link lives inside the "Added!" modal, which only appears after the add-to-cart AJAX call returns. The broken version looks completely harmless when you read it, which is exactly why I chose this defect: it is one of the most common real Selenium mistakes.

7.2 What Happened

Two of the five tests went red, TC04 and TC05, both with the same exception at the same line:

```
org.openqa.selenium.ElementNotInteractableException: element not interactable
(Session info: chrome=150.0.7871.130)
at com.automationexercise.pages.ProductsPage.goToCartFromModal(...)
```

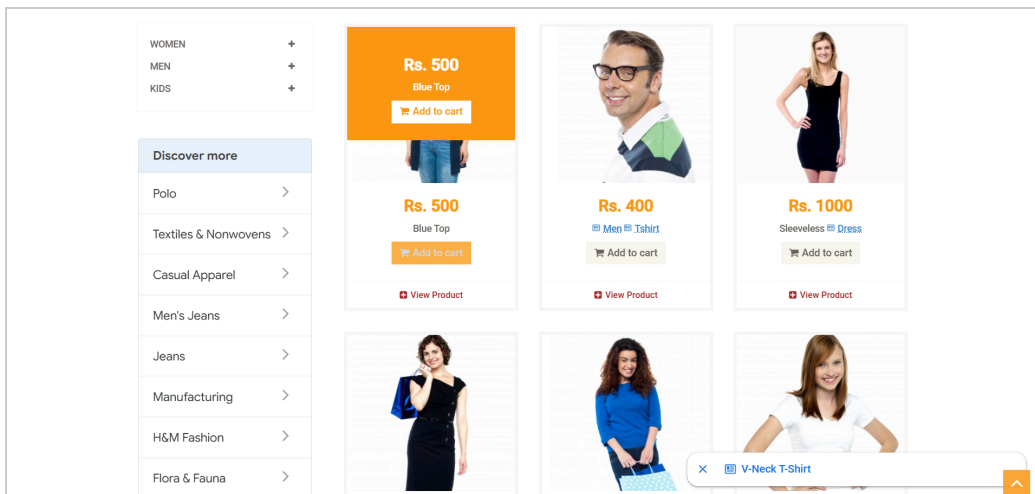


Figure 5: The automatic failure screenshot; the confirmation modal has not opened yet

7.3 How I Debugged It

I started with the stack traces. Two different tests, identical exception, identical method. When several tests fail at one shared method, the problem is almost always in that shared code, not in the tests, so I focused on the page object immediately.

Next I opened the screenshots the listener had captured. Both showed the products page with no modal anywhere. So the click was firing before the modal appeared on screen.

To rule out one test polluting the other, I ran TC04 alone. Same failure every time.

Then I inspected the page in DevTools and found the interesting part: the `#cartModal` element is in the HTML from the moment the page loads. It is just hidden until the AJAX response triggers the fade-in. That explains the exception type, and the exception type was really the biggest clue. Selenium did NOT say `NoSuchElement`. It found the element fine; it just could not click it because it was still invisible. `findElement()` succeeds immediately and has no patience, so my click was racing the AJAX response and losing every time.

7.4 Root Cause

A synchronization defect. Being present in the DOM is not the same as being visible and clickable. Any element that appears through an AJAX response has to be waited for with a condition that describes the state you need, in this case `elementToBeClickable`. The lesson sounds obvious written down, but the defective code reads perfectly fine.

7.5 The Fix

I restored the call to the framework's `WaitUtils.scrollAndClick()`, which waits for clickability before clicking. Rerun: 5 tests, 0 failures.

7.6 A Real Failure in the Middle of It

This part was not planned. While the intentional defect was still in place, TC01 suddenly failed as well, and for a moment I thought my bug had somehow spread. But the exception was a different one:

`ElementClickInterceptedException`, and the trace even named the culprit:

```
element click intercepted: Element <input type="checkbox" id="newsletter" ...>
is not clickable at point (295, 680). Other element would receive the click:
<iframe id="aswift_2" ... src="https://googleads.g.doubleclick.net/..." title="Advertisement">
```

A Google ad banner was sitting exactly on top of the newsletter checkbox on the signup form. This is the ad risk from my requirement analysis actually happening. I fixed it by routing the signup form clicks through the same `scrollAndClick()` helper, which scrolls the element to the middle of the screen where ads cannot cover it. Two different failure types, one shared discipline.

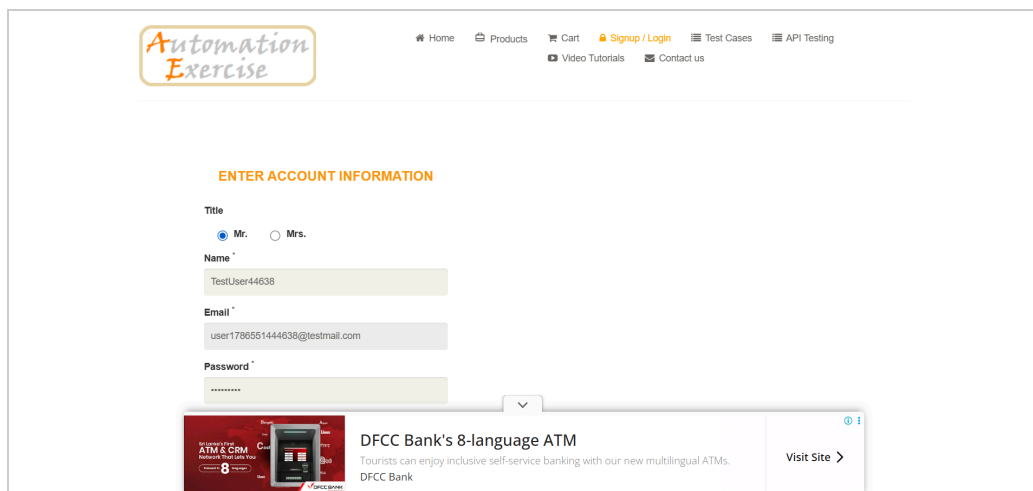


Figure 6: The signup form at the moment the ad iframe intercepted the click

7.7 Lessons Learned

- The exception name tells you where the element got stuck: `NoSuchElement` means not in the DOM, `ElementNotInteractable` means in the DOM but not usable yet, `ElementClickIntercepted` means usable but covered by something. Reading it properly saves a lot of guessing.
- When two tests fail at the same method, look at the method, not the tests. This is a real advantage of the Page Object Model: one fix healed both tests.
- The automatic failure screenshots saved me real time. I could see the modal was missing instead of imagining what the page looked like.
- My search test taught me a related lesson earlier: it first asserted that every result contains the keyword in its name, and failed, because the site also searches product descriptions. Assertions have to encode what the application actually promises, not what I assume it does.

8. Repository and Submission

- **GitHub repository:** https://github.com/UthumImasha/SQA_Assignment. The commit history shows how the project grew step by step, from the documents to the framework to the tests.
- **How to run:** clone the repository and run `mvn test` (JDK 17, Maven and Chrome needed, nothing else).
- **Documentation:** the `docs/` folder holds the full requirement analysis, the 16 scenarios, the automation decision, the framework design and the debugging challenge writeup.
- **AI usage:** declared in `AI_USAGE.md` as the brief requires.